# SIDS Senior Design | Pi 5 Software Handoff

Share with the AI/software teammates. Includes a ready-to-send message, technical requirements, and a prompt for their AI assistant.

## 1. Message to send to the group

Hey everyone! I have the Raspberry Pi 5 and Arducam set up and working. I have tested image capture, 1080p video, and live Python frame capture. I am handling hardware setup and deployment, not developing or modifying the AI model itself.

Could you please provide a complete Raspberry Pi 5 deployment package based on our latest V10 notebook? I need the actual trained model artifacts, MediaPipe assets or verified download links, a standalone live-camera inference script, exact dependencies and setup steps, sample tests, and documentation of the prediction/alarm outputs.

We have a MediaPipe compatibility issue: the Pi runs Debian 13 ARM64 with Python 3.13.5, and we downloaded (but did not install or test) a MediaPipe 0.10.18 wheel for Python 3.11 ARM64. Please work out and document a compatible deployment solution. Also, the current Pi 4 prototype receiver uses TCP port 5000, while your V10 setup proposes UDP port 5005; we need one agreed protocol.

Please keep the training code separate from the Pi deployment code, preserve your validated feature extraction and risk logic, and send me a single ZIP folder that I can install without rewriting your model. I can test the hardware and installation, and I will report any runtime errors back to you. Thank you!

## 2. Existing Pi 5 setup (confirmed working)

- Raspberry Pi 5, 4 GB; Raspberry Pi OS based on Debian 13 (Trixie), aarch64/ARM64.
- Arducam IMX290 CSI camera detected; photos, video, and Picamera2 Python frames tested.
- Python 3.13.5; working virtual environment: ~/senior_design/sids_env.
- OpenCV 4.10.0, scikit-learn 1.4.2, NumPy, and Picamera2 imports tested.
- MediaPipe 0.10.18 Python 3.11 ARM64 wheel downloaded for evaluation only; not installed or validated.

## 3. Required deliverables from the software team

- Trained files: posture_model.joblib, feature_ranges.joblib, risk_cutoffs.joblib, generated by a successful V10 training run. Include any additional files actually required.
- Landmarker assets: pose_landmarker_lite.task and face_landmarker.task, or exact official URLs and integrity/check instructions.
- Standalone inference: monitor_pi5.py that reads frames from Picamera2 and performs the same preprocessing, feature extraction, classification, risk scoring, and temporal smoothing as training.
- Offline test: test_model.py with a few representative test images and expected or reference outputs.
- Environment: pinned requirements, Python version, install commands and compatibility notes for ARM64 Debian 13, including the scikit-learn version used for model export.
- Interface contract: input frame size/color format; exact 12 feature names/order; missing-value behavior; output schema (posture, risk, state, timestamp); alarm transmission protocol/port.

- Documentation: README with folder layout, setup, startup/shutdown, troubleshooting, runtime performance expectations, and known limitations.
- Validation evidence: evaluation results, label definitions, test dataset separation, and examples of errors/uncertain cases. Do not imply clinical validation.

## 4. Important integration issues to resolve

- MediaPipe: the downloaded 0.10.18 wheel targets CPython 3.11, not the installed 3.13; downloading is not proof that imports, dependencies, or Tasks API work.
- Networking: current Pi 4 alarm_receiver.py is a TCP listener on port 5000; V10 setup uses UDP on port 5005. Agree on one approach and test end-to-end.
- Model portability: confirm scikit-learn/joblib compatibility and verify identical feature order and preprocessing. Load joblib files only from trusted teammates.
- Safety: missing landmarks, camera failures, and communication loss must not silently be interpreted as safe. This is an experimental posture-monitoring prototype, not a validated SIDS prevention device.

## 5. Suggested deployment ZIP structure

SIDS_Pi5_Deployment.zip

monitor_pi5.py

test_model.py

requirements.txt

README.md

models/posture_model.joblib

models/feature_ranges.joblib

models/risk_cutoffs.joblib

models/pose_landmarker_lite.task

models/face_landmarker.task

test_images/ (sample images and reference results)

## 6. Prompt to paste into your AI assistant

I am the AI/software developer for a senior-design infant posture monitoring prototype. Review the attached Code_SIDS_project_V10.ipynb and Raspberry Pi setup document. My teammate is responsible for Pi 5 hardware deployment, not editing the ML algorithms. Produce a complete, reproducible Pi 5 deployment handoff based on the actual notebook, preserving the validated feature extraction, classification, and risk-scoring logic unless you identify and clearly explain a defect.

Target hardware: Raspberry Pi 5 (4 GB), Debian 13 Trixie ARM64, Python 3.13.5 system installation, Arducam IMX290 via Picamera2, OpenCV 4.10.0, scikit-learn 1.4.2, and working virtual environment ~/senior_design/sids_env. A MediaPipe 0.10.18 CPython 3.11 ARM64 wheel was downloaded but not installed or tested. The Pi 4 prototype currently listens for TCP JSON messages on port 5000; the V10 setup instead sends UDP JSON to port 5005.

Please: (1) identify every artifact generated by V10 and provide reproducible export steps; (2) create a standalone Picamera2 inference script and offline image test; (3) specify a verified or clearly unverified

ARM64 Python/MediaPipe deployment plan, including version pinning and dependencies; (4) ensure identical feature names/order, color conversion, preprocessing, and missing-value behavior between training and inference; (5) document model outputs and agree on an alarm network protocol/schema; (6) handle no-view, camera, model-loading, and network failures explicitly; (7) provide a README with exact commands and test expectations; (8) report evaluation results and limitations without claiming clinical SIDS detection. Do not fabricate trained .joblib files or claim runtime tests that were not performed. Clearly separate actions you can generate from steps requiring the real training data, model exports, and physical Raspberry Pi testing.

## 7. Definition of ready for hardware handoff

The handoff is ready when the ZIP includes actual trained model artifacts, the deployment script imports in a compatible environment, offline sample-image inference produces documented outputs, and the communication format is agreed. The hardware teammate will then test real Arducam frames and the Pi 4 interface.